

MADRAS INSTITUTE OF TECHNOLOGY

ANNA UNIVERSITY

CHROMEPET, CHENNAI-600044



LABORATORY MANUAL

**EC23S01 NUMERICAL AND SIGNAL PROCESSING PRACTICE
THROUGH PYTHON LABORATORY**

(Regulation – 2023)

DEPARTMENT OF ELECTRONICS ENGINEERING

MADRAS INSTITUTE OF TECHNOLOGY

ANNA UNIVERSITY

LABORATORY MANUAL

EC23S01 NUMERICAL AND SIGNAL PROCESSING PRACTICE THROUGH PYTHON LABORATORY

(Regulation – 2023)

DEPARTMENT OF ELECTRONICS ENGINEERING

MADRAS INSTITUTE OF TECHNOLOGY

ANNA UNIVERSITY

Prepared By

Approved By

Electronics Engineering

Electronics engineering

Madras Institute of Technology

Madras Institute of Technology

VISION

To be recognized as a benchmark and trend setter in Electronics and Communication Engineering domain keeping in phase with rapidly changing technologies through effective partnership with reputed academic institutions, research organizations, industries and community.

MISSION

- Create highly motivated, technologically competent human resource by imparting high quality technical education through flexible student centric updated curricula suited to students with diverse backgrounds.
- Adopt best teaching and learning practices and establish state-of-the-art facilities to provide quality academic ambience for innovativeness, research and developmental activities.
- Enhance collaborative activities with academic institutions and industries for evolving indigenous technological solutions to meet societal needs and nurture leadership and entrepreneurship qualities with ethical means.
- Facilitate adequate exposure to the students, faculty and staff through training in the state-of-the art technologies, efficient administration, global outreach and benchmarking against referential institutions.

PROGRAMME EDUCATIONAL OBJECTIVES (PEOs):

PEO1: The graduated Students will demonstrate sufficient theoretical, analytical and initiative skills in Basic Sciences and Engineering necessary, to assimilate, analyze, synthesize and innovative solutions to meet societal needs.

PEO2: The graduated students will have inculcated a thirst for lifelong learning and sustained research interest.

PEO 3: The graduated students will practice values and exhibit leadership qualities and team spirit to promote entrepreneurship and indigenization.

PROGRAMME OUTCOMES (POs):

PO1: Ability to apply technical knowledge in mathematics, Science and Engineering leading to the realization and evaluation of complex systems, through research problems in the context of evolving societal needs.

PO2: Imaginative critical thinking with an ability to think critically, analyze and solve Engineering problems.

PO3: Ability to design a system, component, or process to meet desired needs within realistic constraints.

PO4: Ability to, gather user needs and requirements, design, develop, integrate, and test complex systems by employing systems engineering thinking and processes, within required operational and acquisition system environments.

PO5: Personal and intellectual autonomy to independently and with an openness to reflect upon and use modern engineering tools necessary to engineering practices.

PO6: Educational practices necessary to understand the impact of engineering solutions in a global, economical, environmental and societal context.

PO7: An active and committed global citizen with an awareness of contemporary issues and their Impact on economic, environmental, social, political, ethical, health and safety, manufacturability and sustainability.

PO8: An understanding of professional, ethical, legal issues and responsibilities.

PO9: A creative, enterprising team player and engaged participative leader able to effect change.

PO10: A confident, resilient and adaptable individual with good communication skills.

PO11: Exercise their responsibilities in the management of cost-effective systems product development by leading and participating in interdisciplinary teams.

PO12: Active exploration of new ideas through lifelong learning.

Course Outcomes:**CO1: Develop and execute simple Python programs****CO2: Apply the best features available in Python to solve the situational problems****CO3: Ability to develop and analyze wireless network securing protocol****CO4: Develop code for numerical methods computation using Python****CO5: Generate and analyze the basic signals using Python****Course Articulation Matrix:**

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	3	3	3	3	3						2	2
CO2	3	3	3	3	3						2	2
CO3	3	3	3	3	3						1	2
CO4	3	3	3	3	3	2					1	1

LIST OF EXPERIMENTS:

1. Getting Started with Python and using the Plot command
2. Load data from files and Plotting data
3. Getting Started with Lists
4. Getting started with for, If, While loops
5. Getting started with files and arrays
6. Statistics using Python
7. Linear combination of vectors and Computation of determinant, rank of a matrix
8. Lower - Upper Decomposition of vectors
9. Gauss-Seidel Method
10. Solve Systems of Linear Equations in Python
11. Eigenvalues and eigenvectors in Python
12. Generation of basic sequences using Python
13. Spectral analysis of signals
14. Sampling of continuous-time signals

Exp. No.: 1	Getting Started with Python and using the Plot command	Date:
-------------	--	-------

Aim:

To introduce Python for signal processing applications and to learn how to generate and visualize a basic sine wave using numpy and matplotlib

Tools required:

- Python (3.x)

Theory:

In signal processing, a sine wave is one of the most fundamental continuous-time signals. It is defined as:

$$x(t) = A \cdot \sin(2\pi f t + \phi)$$

Procedure:

1. Import required libraries: numpy for numeric operations, matplotlib.pyplot for plotting.
2. Create a time vector from 0 to 1 second.
3. Generate a sine wave of frequency 5 Hz.
4. Plot the sine wave using plt.plot().

Continuous-Time Signal

- (a) Plot a sine wave $x(t) = \sin(2\pi f t)$

Program:

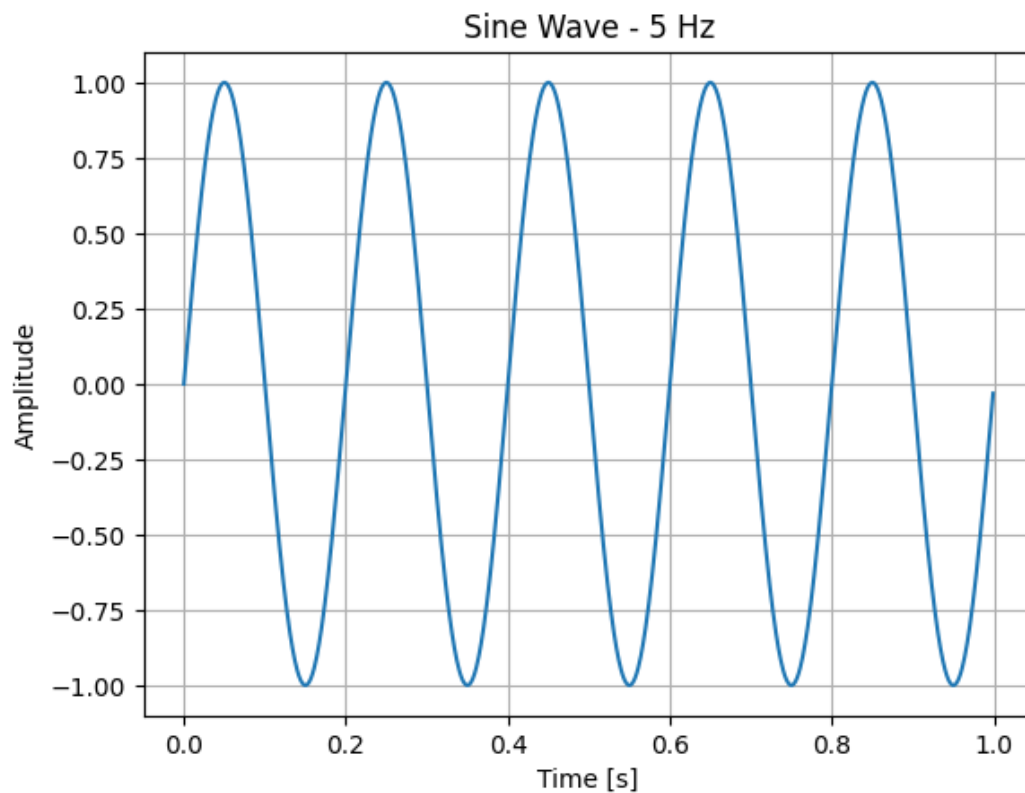
```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

# Create a time array from 0 to 1 second, with 1000 samples
t = np.linspace(0, 1, 1000, endpoint=False)

# Create a sine wave signal
freq = 5 # 5 Hz
x = np.sin(2 * np.pi * freq * t)

# Plot the signal
plt.plot(t, x)
plt.title("Sine Wave - 5 Hz")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```

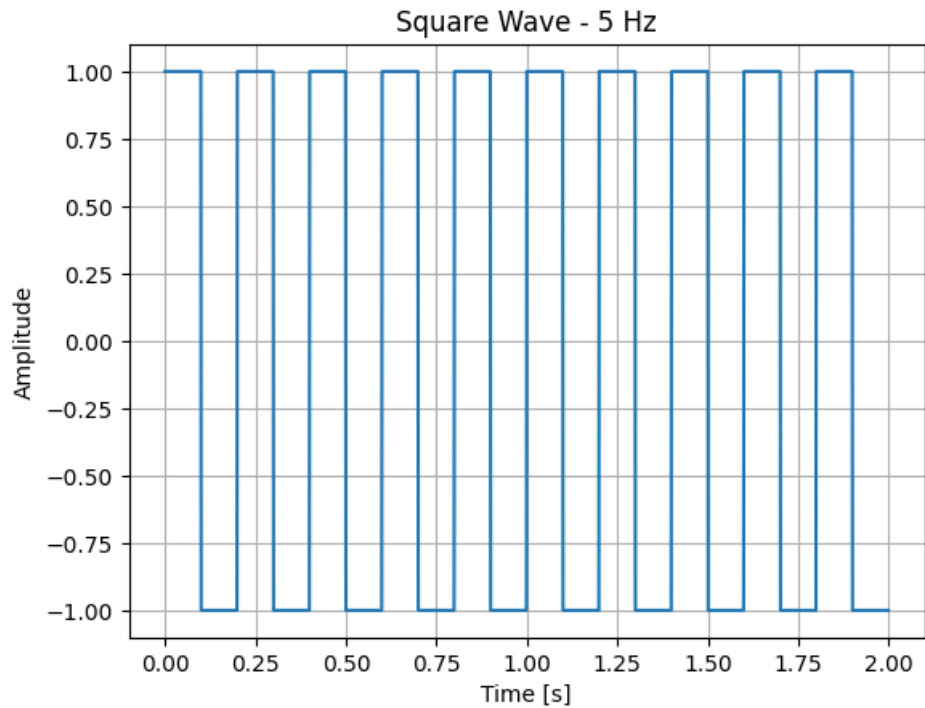

Output:



(b) Square Wave

```
import numpy as np
import matplotlib.pyplot as plt
x_square = signal.square(2 * np.pi * freq * t)

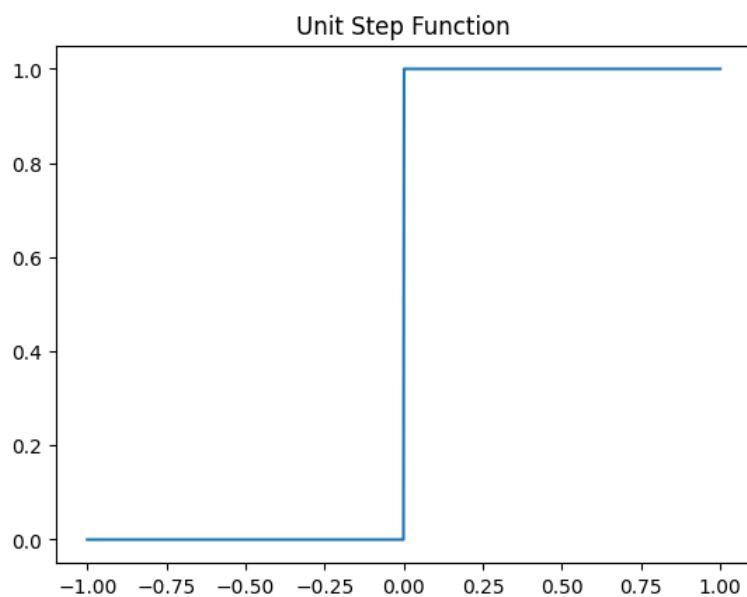
plt.plot(t, x_square)
plt.title("Square Wave - 5 Hz")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```



(c) Unit Step Function

```
import numpy as np
import matplotlib.pyplot as plt
x_square = signal.square(2 * np.pi * freq * t)

t = np.linspace(-1, 1, 1000)
x = np.heaviside(t, 1)
plt.plot(t, x)
plt.title("Unit Step Function")
```



Theory:

1. Scaling a Signal

To multiply each sample of a signal $x[n]$ by a constant a :

$$y[n] = a \cdot x[n]$$

2. Signal Addition

Adding two signals $x[n]$ and $y[n]$

$$z[n] = x[n] + y[n]$$

3. Reversing a Signal

To reverse a signal $x[n] \rightarrow x[-n]$

$x_{rev}[n] = x[-n]$ (represented by reversing the list)

4. Shifting a Signal

- **Right Shift by k:** $y[n] = x[n-k]$
- **Left Shift by k:** $y[n] = x[n+k]$

Procedure:

- Create a time vector n using `range()` or a list.
- Define discrete-time signals (step, ramp) using list comprehensions.
- Apply signal operations: scaling, addition, reversal, shifting.
- Use `matplotlib.pyplot.stem()` to plot signals.
- Interpret the behavior of each signal and operation.

```
import matplotlib.pyplot as plt

# 1. Creating basic signals using lists
n = list(range(0, 10)) # Time indices

# Unit step signal
unit_step = [1 for i in n]

# Ramp signal
ramp = [i for i in n]

# 2. Scaling a signal
scaled_ramp = [2 * x for x in ramp]

# 3. Adding two signals
combined_signal = [u + r for u, r in zip(unit_step, ramp)]
```

4. Reversing a signal

```
reversed_ramp = ramp[::-1]
```

5. Shifting a signal (Right shift by 2)

```
right_shifted = [0, 0] + ramp[:-2]
```

Plotting all signals

```
plt.figure(figsize=(12, 8))
```

```
plt.subplot(3, 2, 1)
```

```
plt.stem(n, unit_step)
```

```
plt.title("Unit Step Signal")
```

```
plt.xlabel("n"); plt.ylabel("Amplitude")
```

```
plt.subplot(3, 2, 2)
```

```
plt.stem(n, ramp)
```

```
plt.title("Ramp Signal")
```

```
plt.subplot(3, 2, 3)
```

```
plt.stem(n, scaled_ramp)
```

```
plt.title("Scaled Ramp Signal")
```

```
plt.subplot(3, 2, 4)
```

```
plt.stem(n, combined_signal)
```

```
plt.title("Combined Signal (Step + Ramp)")
```

```
plt.subplot(3, 2, 5)
```

```
plt.stem(n, reversed_ramp)
```

```
plt.title("Reversed Ramp Signal")
```

```
plt.subplot(3, 2, 6)
```

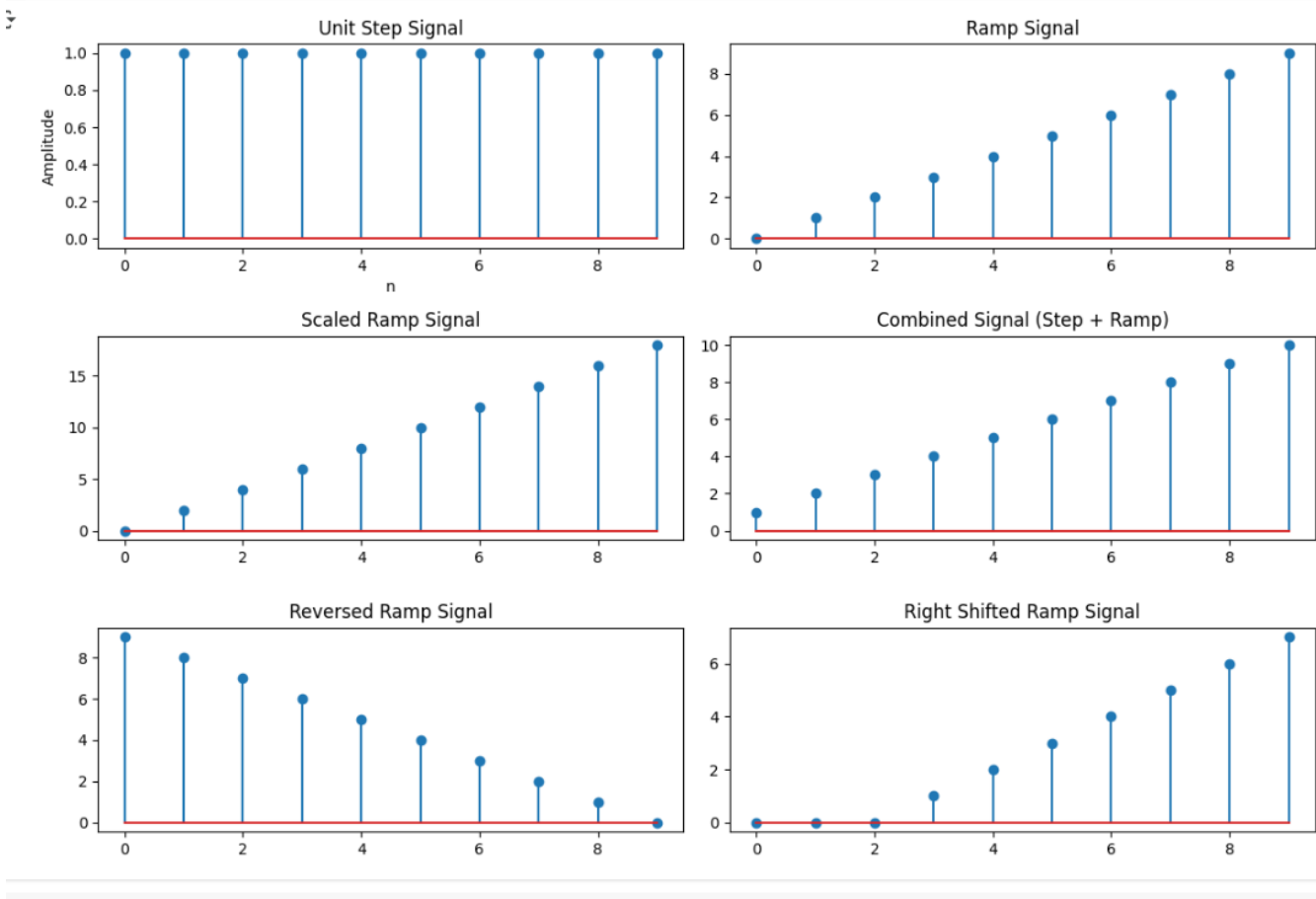
```
plt.stem(n, right_shifted)
```

```
plt.title("Right Shifted Ramp Signal")
```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT:



Result:

A time-domain waves were generated and plotted using Python successfully completed

Exp. No.: 2	Load data from files and Plotting data	Date:
-------------	--	-------

Aim:**Tools required:**

- Python (3.x)

Theory:

To load data from files and plot it for signal processing analysis, the procedure typically involves the following steps in Python (commonly using NumPy, Pandas, and Matplotlib or SciPy).

Procedure:

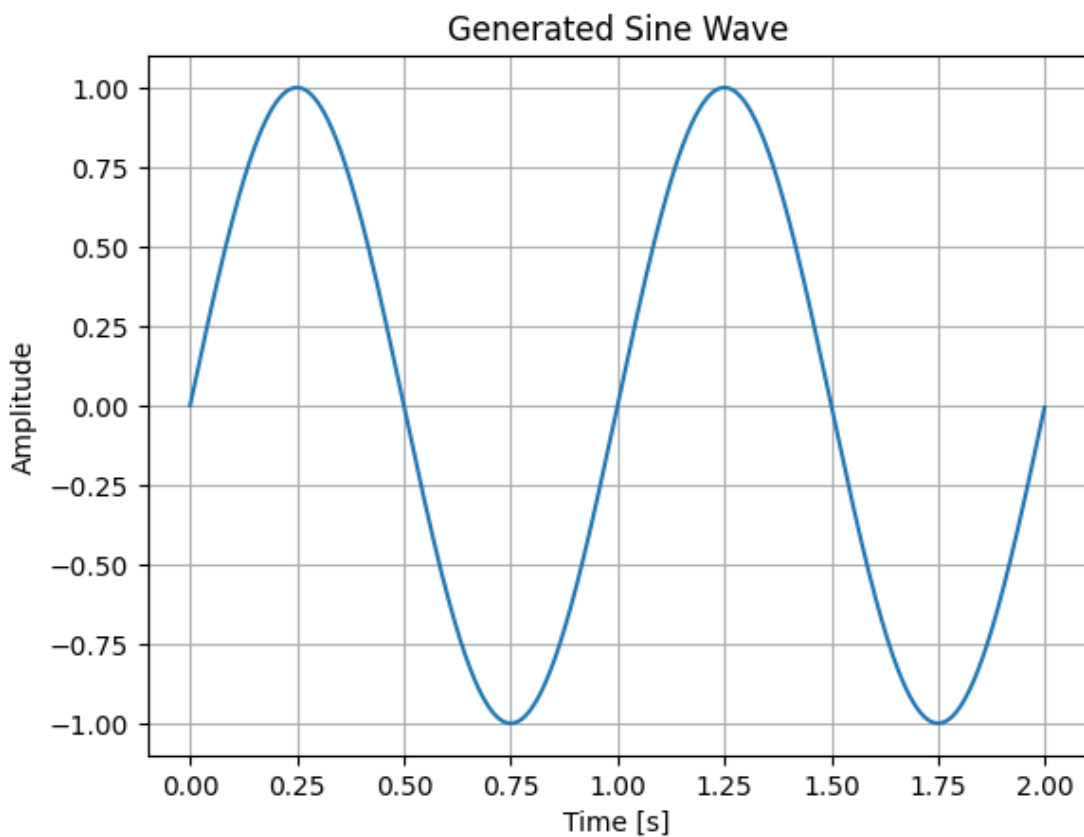
(a) sine wave

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters
frequency = 1 # Hz
amplitude = 1
duration = 2 # seconds
sampling_rate = 1000 # Hz

# Generate data
t = np.linspace(0, duration, int(sampling_rate * duration), endpoint=False)
y = amplitude * np.sin(2 * np.pi * frequency * t)

# Plot
plt.plot(t, y)
plt.title('Generated Sine Wave')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.grid(True)
plt.show()
```



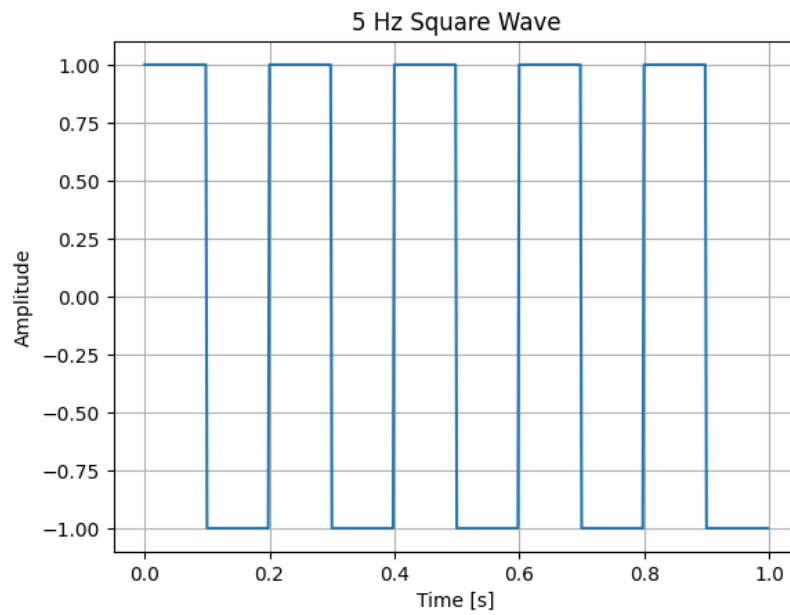
(b) Square Wave

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

# Time vector
t = np.linspace(0, 1, 500, endpoint=False)

# Generate square wave
square_wave = signal.square(2 * np.pi * 5 * t) # 5 Hz frequency

# Plotting
plt.plot(t, square_wave)
plt.title("5 Hz Square Wave")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```



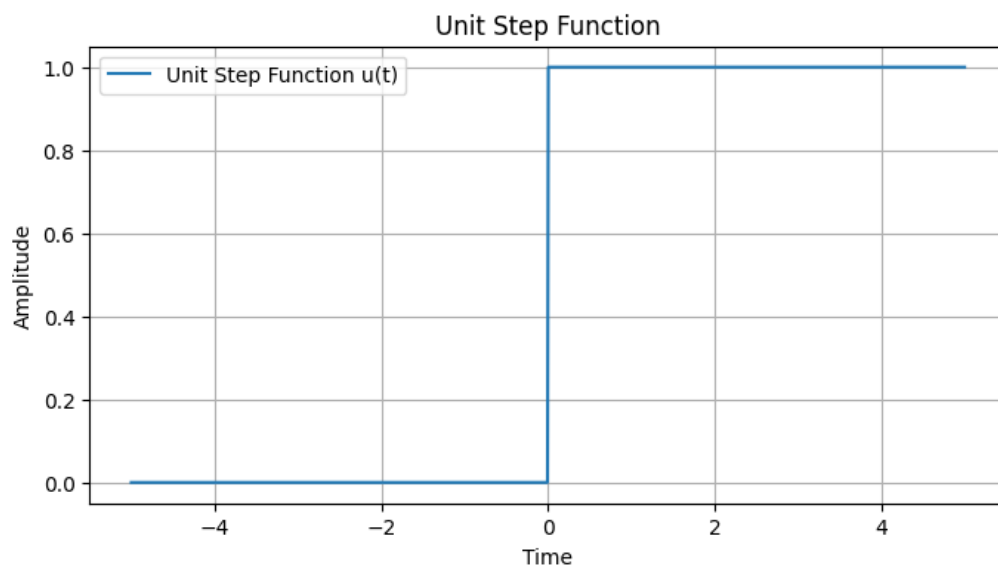
(c) Unit Step Function

```
import numpy as np
import matplotlib.pyplot as plt

# Time range
t = np.linspace(-5, 5, 1000)

# Unit step function
u = np.where(t >= 0, 1, 0)

# Plot
plt.figure(figsize=(8, 4))
plt.plot(t, u, label='Unit Step Function u(t)')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.title('Unit Step Function')
plt.grid(True)
plt.legend()
plt.show()
```

Result:

Thus the Load data from files and Plotting data using python successfully

Exp. No.: 3	Getting Started with Lists	Date:
-------------	----------------------------	-------

Aim:

Write a Python program using list to perform and visualize the convolution of two discrete-time Use a rectangular pulse $x_1[n]$ and an exponential decay signal $x_2[n]=e^{-0.5n}u[n]$. Plot the two signals and their convolution

Tools required:

- Python (3.x)

Theory:

Getting Started with Lists

Convolution of Discrete-Time Signals

Write a Python program to perform and visualize the convolution of two discrete-time Use a rectangular pulse $x_1[n]$ and an exponential decay signal $x_2[n]=e^{-0.5n}u[n]$. Plot the two signals and their convolution.

```
import math
import matplotlib.pyplot as plt

# Define n as a list
n = list(range(0, 10))

# Rectangular pulse x1[n] = 1 for n=0 to 4
x1 = list([1] * 5)

# Exponential decay signal x2[n] = exp(-0.5 * n)
x2 = [math.exp(-0.5 * i) for i in n]

# Perform linear convolution manually using lists
m = len(x1)
n2 = len(x2)
conv_result = list([0] * (m + n2 - 1))

for i in range(m + n2 - 1):
    for j in range(m):
        if 0 <= i - j < n2:
            conv_result[i] += x1[j] * x2[i - j]

# Plotting
plt.figure(figsize=(10, 6))

# x1[n]
plt.subplot(3, 1, 1)
plt.stem(range(len(x1)), x1, linefmt='b-', markerfmt='bo', basefmt="
", label='Rectangular Pulse')
plt.legend()

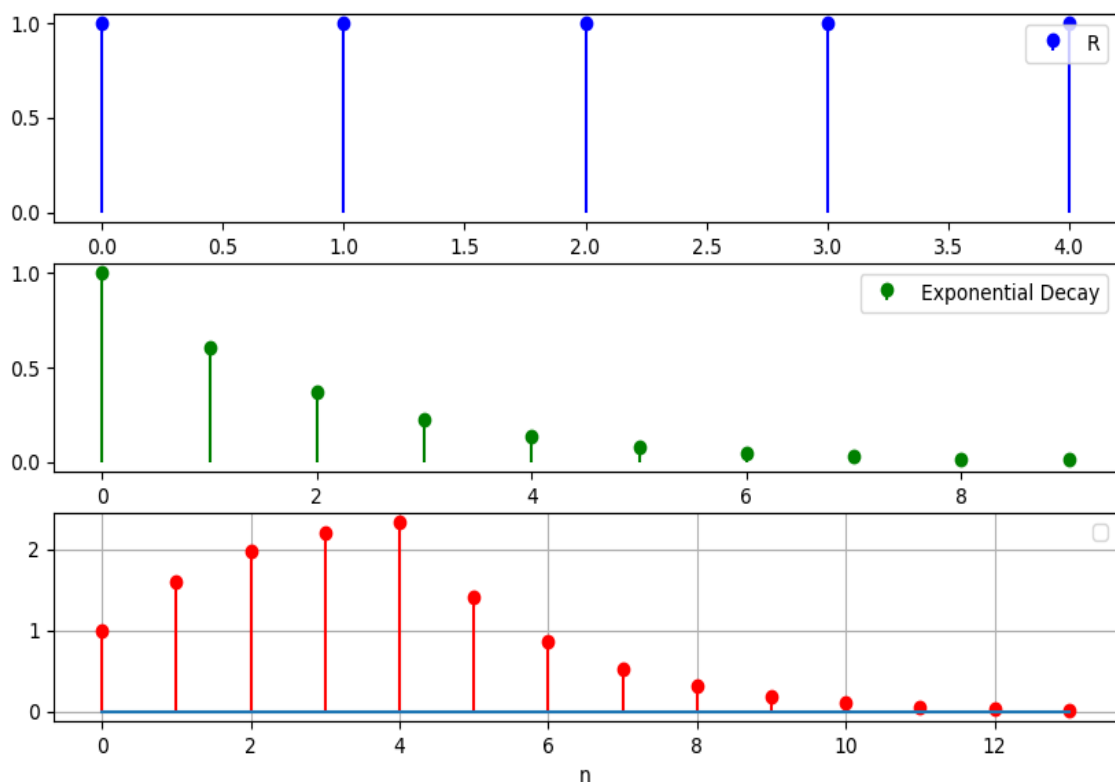
# x2[n]
plt.subplot(3, 1, 2)
```

```
plt.stem(range(len(x2)), x2, linefmt='g-', markerfmt='go', basefmt="
", label='Exponential Decay')
plt.legend()

# Convolution result
plt.subplot(3, 1, 3)
plt.stem(range(len(conv_result)), conv_result, linefmt='r-',
markerfmt='ro', basefmt="", label='Convolution Result')
plt.legend()

plt.xlabel('n')
plt.grid(True)
plt.tight_layout()
plt.show()
```

OUTPUT:



Convolution of Discrete-Time Signals - without Inbuilt Function

CODE: - Linear Convolution

```
import matplotlib.pyplot as plt

# Generate Input Signals using list()
x = list([1, 2, 3, 4])
h = list([1, 1, 1, -1])

# Lengths
m = len(x)
```

```

n = len(h)
# Zero Padding using list() and +
X = x + list([0]*n)
H = h + list([0]*m)
# Convolution Operation using lists
Y = list([0]*(m + n - 1))
for i in range(m + n - 1):
    for j in range(m):
        if 0 <= i - j < n + m:
            Y[i] += X[j] * H[i - j]
# Plot results
plt.figure(figsize=(10, 6))

plt.subplot(3, 1, 1)
plt.stem(range(len(x)), x, linefmt='b-', markerfmt='b^', basefmt='k')
plt.xlabel('n')
plt.ylabel('x[n]')
plt.grid(True)

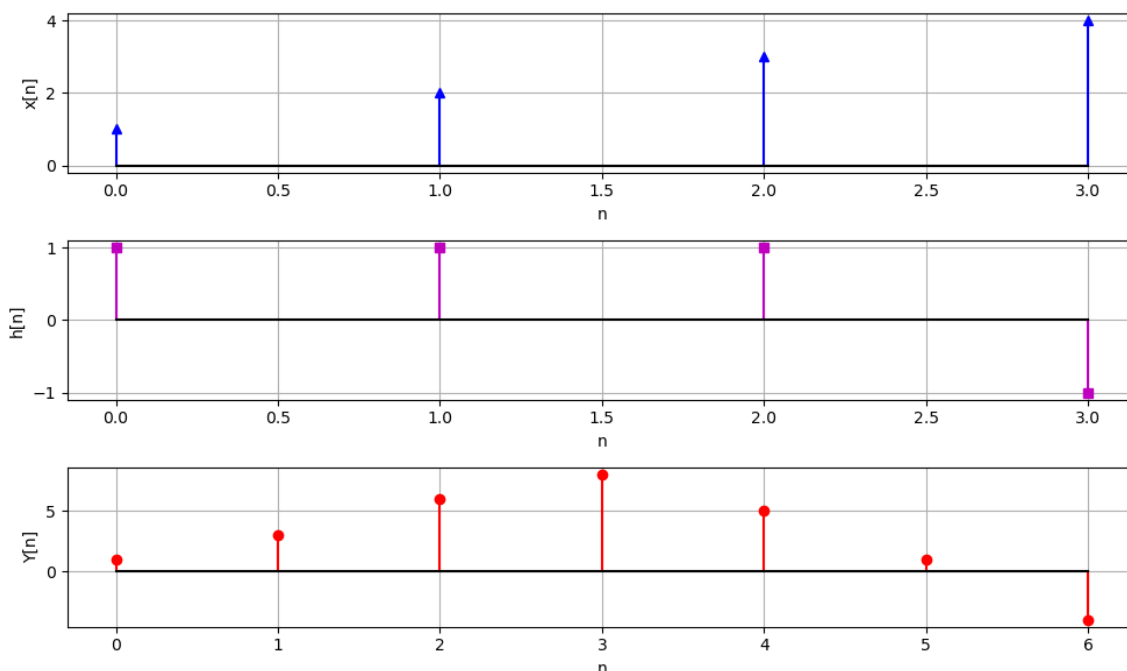
plt.subplot(3, 1, 2)
plt.stem(range(len(h)), h, linefmt='m-', markerfmt='ms', basefmt='k')
plt.xlabel('n')
plt.ylabel('h[n]')
plt.grid(True)

plt.subplot(3, 1, 3)
plt.stem(range(len(Y)), Y, linefmt='r-', markerfmt='ro', basefmt='k')
plt.xlabel('n')
plt.ylabel('Y[n]')
plt.grid(True)

plt.tight_layout()
plt.show()

```

OUTPUT:



CODE: - Circular Convolution

```
import numpy as np
import matplotlib.pyplot as plt

# Input sequences using list()
x = list([1, 2, 3, 4])
h = list([1, 1, 1, -1])

# Length for circular convolution = max(len(x), len(h))
N = max(len(x), len(h))

# Zero pad both sequences to length N using list() and +
x_padded = x + list([0] * (N - len(x)))
h_padded = h + list([0] * (N - len(h)))

# Perform FFT-based circular convolution
X = np.fft.fft(x_padded)
H = np.fft.fft(h_padded)
Y = np.fft.ifft(X * H).real # Keep real part

# Convert result to list
Y = list(Y)

# Plot results
plt.figure(figsize=(10, 6))

plt.subplot(3, 1, 1)
plt.stem(range(len(x)), x, linefmt='b-', markerfmt='b^', basefmt='k')
plt.xlabel('n')
plt.ylabel('x[n]')
plt.grid(True)

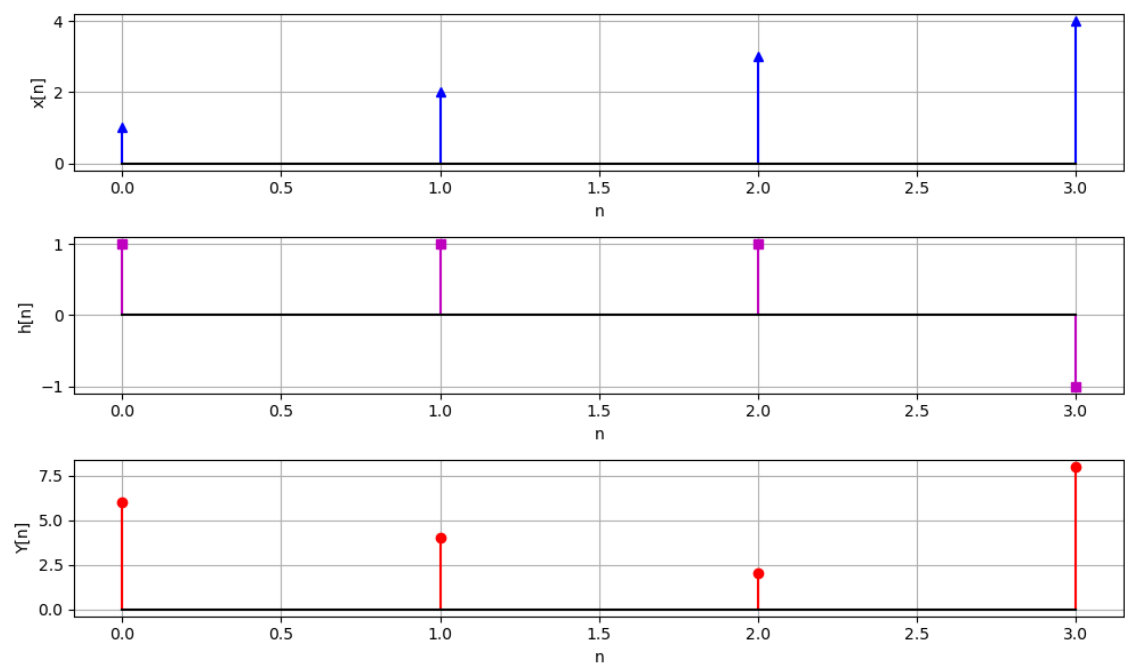
plt.subplot(3, 1, 2)
plt.stem(range(len(h)), h, linefmt='m-', markerfmt='ms', basefmt='k')
plt.xlabel('n')

plt.ylabel('h[n]')
plt.grid(True)

plt.subplot(3, 1, 3)
plt.stem(range(len(Y)), Y, linefmt='r-', markerfmt='ro', basefmt='k')
plt.xlabel('n')
plt.ylabel('Y[n]')
plt.grid(True)

plt.tight_layout()
plt.show()
```

OUTPUT:



RESULT

Thus the Getting Started with Lists using python successfully

Exp. No.: 4	Getting started with for, If, While loops	Date:
-------------	---	-------

Fourier Transform of Basic Signals

Write a Python program to compute the Discrete Fourier Transform (DFT) of a sinusoidal signal $x(t)=\sin(2\pi 10t)$. Use `numpy.fft` to perform the Fourier Transform and plot both the time-domain signal and the magnitude spectrum. Discuss how the frequency content is reflected in the spectrum.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 1, 1000)
x = np.sin(2 * np.pi * 100 * t)+np.sin(2 * np.pi * 200 * t)

X_f = np.fft.fft(x)

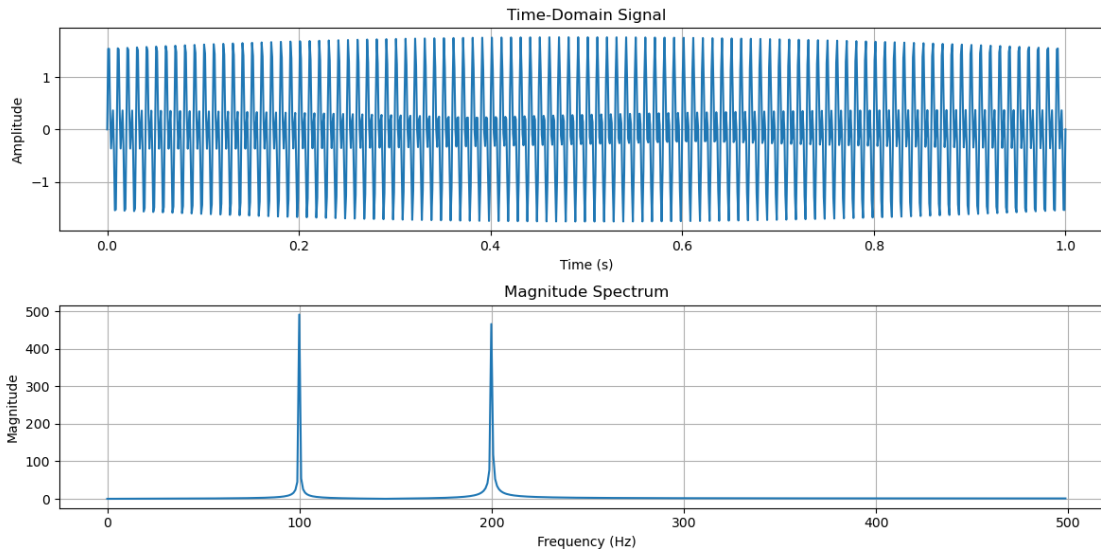
frequencies = np.fft.fftfreq(len(t), d=t[1] - t[0])
magnitude_spectrum = np.abs(X_f)

plt.figure(figsize=(12, 6))
plt.subplot(2, 1, 1)
plt.plot(t, x, label='x(t) = sin(2π10t)')
plt.xlabel("Time (s)")

plt.ylabel('Amplitude')
plt.title("Time-Domain Signal")
plt.grid(True)

plt.subplot(2, 1, 2)
plt.plot(frequencies[:len(frequencies)//2], magnitude_spectrum[:len(frequencies)//2])
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude')
plt.title('Magnitude Spectrum')
plt.grid(True)
plt.tight_layout()
plt.show()
```

OUTPUT:



CODE: Using FOR statement

```
import numpy as np

# Sample input signal
x = [1, 2, 3, 4] # You can use any real or complex signal
N = len(x)
X = []

# DFT formula:
X[k] = sum_n=0_to_N-1 x[n] * exp(-j*2*pi*k*n/N)
for k in range(N):
    real = 0
    imag = 0
    for n in range(N):

        angle = 2 * np.pi * k * n / N
        real += x[n] * np.cos(-angle)
    imag += x[n] * np.sin(-angle)
    X.append(complex(real, imag))

# Print the DFT result
print("DFT of x[n]:")
for i, val in enumerate(X):
    print(f"X[{i}] = {val}")
```

OUTPUT:

```
DFT of x[n]:
X[0] = (10+0j)
X[1] = (-2.0000000000000004+1.9999999999999996j)
X[2] = (-2-9.797174393178826e-16j)
X[3] = (-1.9999999999999982-2.000000000000001j)
```

Sampling with Noise and Filtering

Sample a 5 Hz sinusoidal signal corrupted by white Gaussian noise at 10 Hz, 20 Hz, a Apply a low-pass filter to the noisy signal and reconstruct the clean signal. How does the sampling rate impact the filtering and noise reduction performance?

CODE: - Using FOR Statement


```

import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, filtfilt

# Parameters
fs_list = [10, 20, 50]
f, duration, noise_std = 5, 2, 0.5
cutoff = 3

# Define low-pass filter
def lowpass_filter(data, cutoff, fs):
    b, a = butter(4, cutoff / (0.5 * fs), btype='low')
    return filtfilt(b, a, data)

# High-resolution time axis
time_fine = np.linspace(0, duration, 1000)
clean_signal = np.sin(2 * np.pi * f * time_fine)

# Plotting
plt.figure(figsize=(10, 8))

for i, fs in enumerate(fs_list):
    # Coarse sampling for noisy signal
    time = np.linspace(0, duration, int(fs * duration), endpoint=False)
    noisy_signal = np.sin(2 * np.pi * f * time) + np.random.normal(0, noise_std, len(time))
    filtered_signal = lowpass_filter(noisy_signal, cutoff, fs)

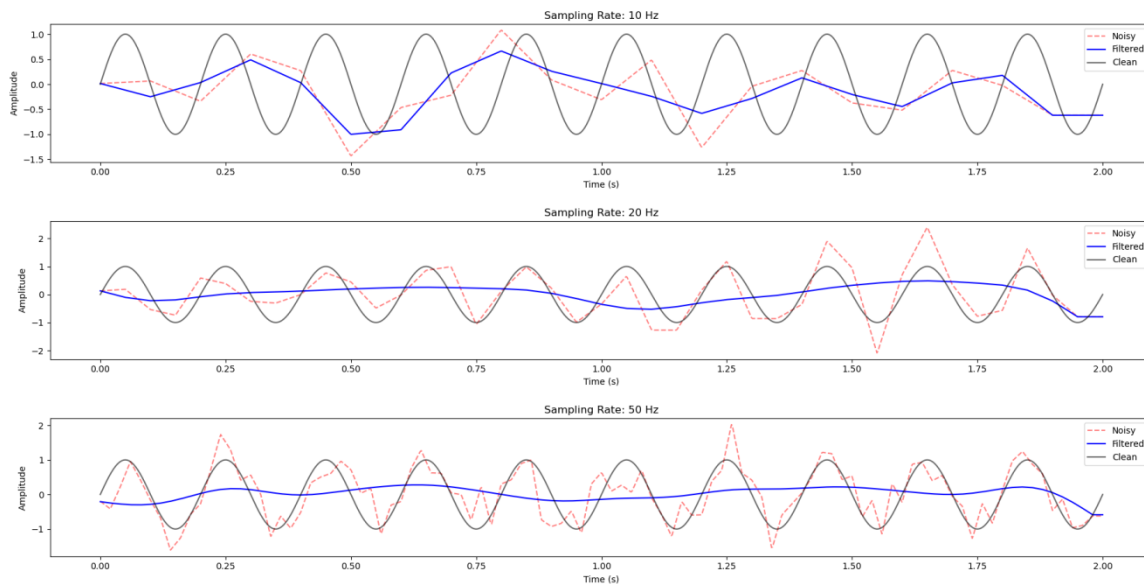
    # Interpolate to high-resolution time base
    noisy_interp = np.interp(time_fine, time, noisy_signal)
    filtered_interp = np.interp(time_fine, time, filtered_signal)

    # Plot
    plt.subplot(len(fs_list), 1, i + 1)
    plt.plot(time_fine, noisy_interp, 'r--', alpha=0.5, label='Noisy')
    plt.plot(time_fine, filtered_interp, 'b', label='Filtered')
    plt.plot(time_fine, clean_signal, 'k', alpha=0.6, label='Clean')

    plt.title(f"Sampling Rate: {fs} Hz")
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude")
    plt.legend(loc="upper right")
    plt.tight_layout()
    plt.show()

```

OUTPUT:



Clipping Signal Using IF statement

CODE:

```
# Original signal (can be any list of values)
signal = [1, 3, 7, 12, 5, -4, -10, 2]
```

```
# Clipping thresholds
```

```
min_val = -5
```

```
max_val = 10
```

```
# Clipped signal (using 'if')
```

```
clipped_signal = []
```

```
for sample in signal:
```

```
    if sample > max_val:
```

```
        clipped_signal.append(max_val)
```

```
    elif sample < min_val:
```

```
        clipped_signal.append(min_val)
```

```
    else:
```

```
        clipped_signal.append(sample)
```

```
print("Original Signal: ", signal)
```

```
print("Clipped Signal: ", clipped_signal)
```

OUTPUT:

```
Original Signal: [1, 3, 7, 12, 5, -4, -10, 2]
Clipped Signal:  [1, 3, 7, 10, 5, -4, -5, 2]
```

Signal Normalization using WHILE Statement:

CODE:

#Signal normalization typically means scaling all values so they lie between 0 and 1, or sometimes between -1 and 1

```
# Original signal
signal = [4, 8, 15, 16, 23, 42]

# Find min and max
min_val = min(signal)
max_val = max(signal)

# Normalized signal (initialize)
normalized_signal = []

# Counter for while loop
i = 0
while i < len(signal):
# Apply normalization formula
    norm = (signal[i] - min_val) / (max_val - min_val)
    normalized_signal.append(norm)
    i += 1

# Display results
print("Original Signal: ", signal)
print("Normalized Signal: ", normalized_signal)
```

OUTPUT:

```
Original Signal:  [4, 8, 15, 16, 23, 42]
Normalized Signal:  [0.0, 0.10526315789473684, 0.2894736842105263, 0.3157894736842105, 0.5, 1.0]
```

Result:

The solution to the system of equations using the Getting started with for, If, While loops was successfully obtained.

Exp. No.: 5	Getting started with files and arrays Aliasing in Real-World Signals	Date:
-------------	--	-------

Load an audio signal using `librosa` or `scipy.io.wavfile`, and downsample it to different rates (e.g., 8 kHz, 16 kHz, 44.1 kHz). Play the audio files and compare the quality. At what point does aliasing become noticeable? Plot the frequency-domain representation to show how the signal spectrum changes with the sampling rate.

CODE: Using `scipy.io.wavfile`

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import resample

# Original signal parameters
sr = 44100
t = np.linspace(0, 1, sr, endpoint=False)

original_signal = np.sin(2 * np.pi * 440 * t) + 0.5 * np.sin(2 * np.pi * 880 * t)

# Downsampling rates
downsampling_rates = [8000, 16000, 44100]

# Create figure
plt.figure(figsize=(12, 8))

for idx, rate in enumerate(downsampling_rates):
    # Downsample the signal
    downsampled_signal = resample(original_signal, rate)

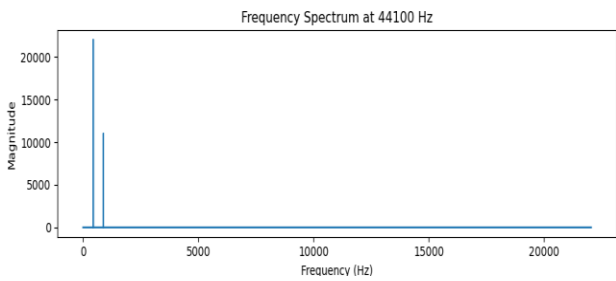
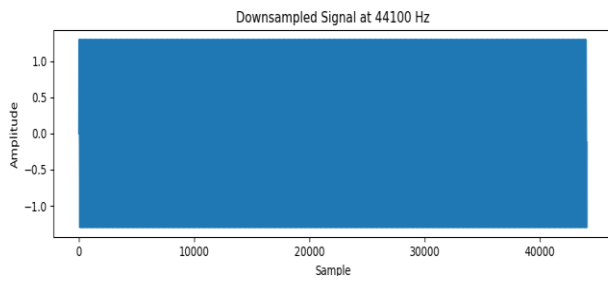
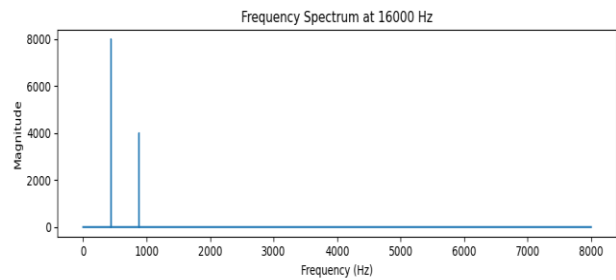
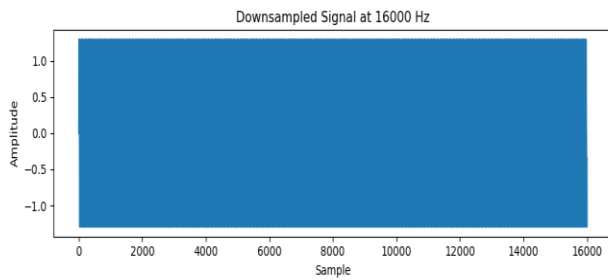
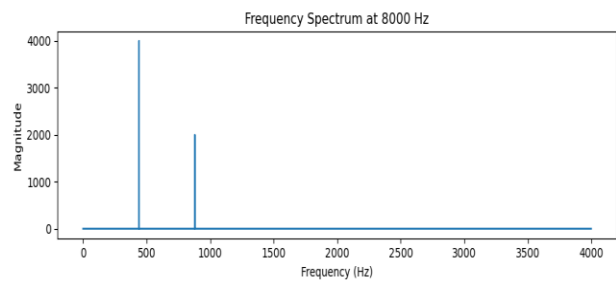
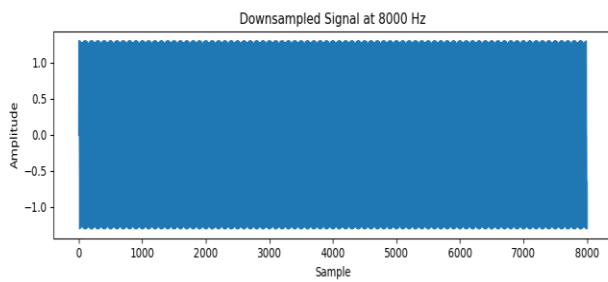
    # Compute FFT and frequency axis
    frequencies = np.fft.fftfreq(len(downsampled_signal), d=1/rate)
    Y_f = np.fft.fft(downsampled_signal)

    # Plot time-domain signal
    plt.subplot(3, 2, 2 * idx + 1)
    plt.plot(downsampled_signal)
    plt.title(f'Downsampled Signal at {rate} Hz')
    plt.xlabel('Sample')
    plt.ylabel('Amplitude')

    # Plot frequency-domain spectrum
    plt.subplot(3, 2, 2 * idx + 2)
    plt.plot(frequencies[:len(frequencies)//2], np.abs(Y_f)[:len(Y_f)//2])
    plt.title(f'Frequency Spectrum at {rate} Hz')
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('Magnitude')

# Final layout adjustment
plt.tight_layout()
plt.show()
```

OUTPUT:



CODE: Using librosa

```
import librosa
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Audio

# Load audio file (example trumpet sound)
signal, sr = librosa.load(librosa.ex('trumpet'))

# Function to plot frequency spectrum
def plot_spec(sig, sr, tle="Spectrum"):
    D = np.abs(np.fft.fft(sig))
    freqs = np.fft.fftfreq(len(D), 1/sr)

    plt.figure(figsize=(8, 4))
    plt.plot(freqs[:len(freqs)//2], D[:len(D)//2])
    plt.title(tle) # Fixed typo from 'tle'
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Amplitude")
    plt.grid(True)
    plt.show()

# Plot spectrum of original signal
plot_spec(signal, sr, "Original Signal Spectrum")

# Downsample rates
rates = [8000, 16000, 44100]
```

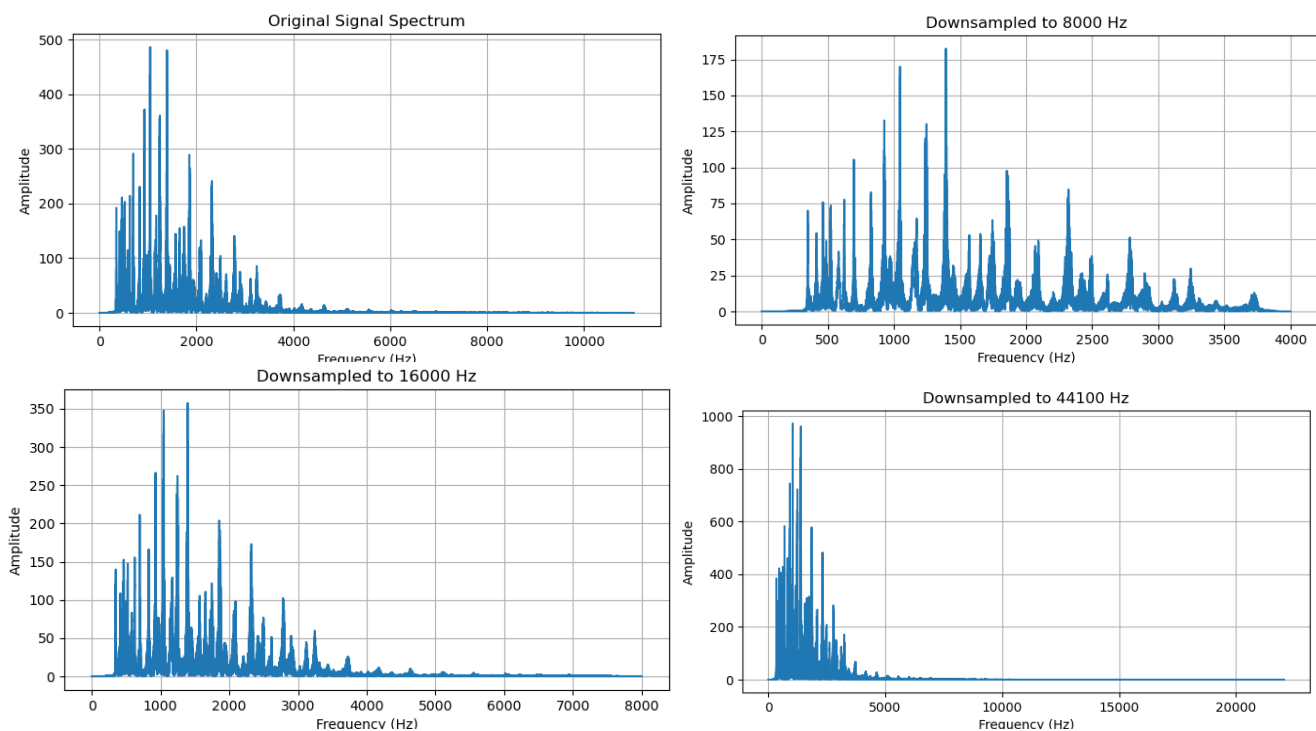
```
# Downsample, plot spectrum, and play audio
```

```
for r in rates:
```

```
ds_signal = librosa.resample(y=signal, orig_sr=sr, target_sr=r)
```

```
plot_spec(ds_signal, r, f'Downsampled to {r} Hz')
```

OUTPUT:



Power Spectral Density (PSD) - Using Numpy Array

Write a Python program to compute the Power Spectral Density (PSD) of a noisy signal consisting of a 5 Hz sinusoid and Gaussian noise. Use the matplotlib library to plot the PSD and explain how the noise and signal components appear in the frequency

CODE:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from scipy.signal import welch
```

```
# Sampling settings
```

```
fs = 100      # Sampling frequency (Hz)
```

```
t_duration = 2 # Duration in seconds
```

```
f_signal = 5   # Frequency of the sine wave (Hz)
```

```
# Time axis - creates a 1D NumPy array of time points
```

```
t = np.arange(0, t_duration, 1/fs)
```

```
# Signal + noise
```

```

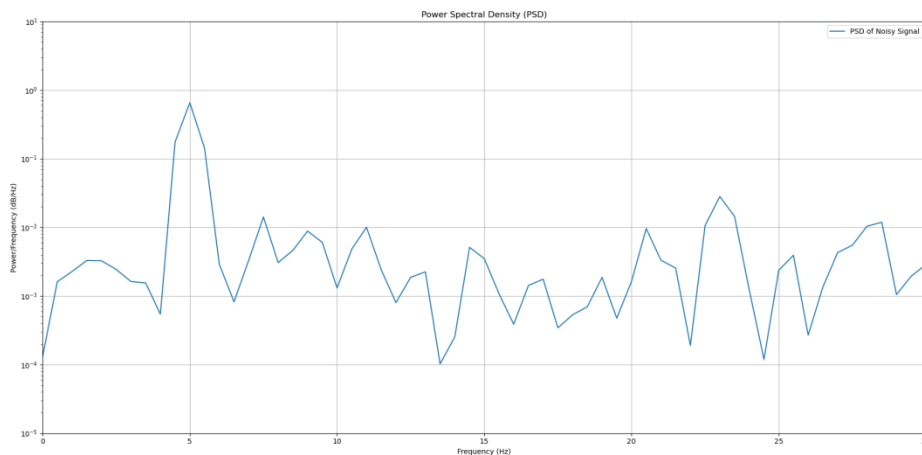
# Creates a sine wave using the time array t
signal = np.sin(2 * np.pi * f_signal * t)
# Generates Gaussian noise as a NumPy array of the same length as t
noise = np.random.normal(0, 0.5, len(t))
# Element-wise addition of two arrays (signal + noise).
noisy_signal = signal + noise

# Power Spectral Density using Welch method
frequencies, psd = welch(noisy_signal, fs, nperseg=256)

# Plot
plt.figure(figsize=(10, 6))
# Plotting two arrays: frequency values vs. power values.
plt.semilogy(frequencies, psd, label='PSD of Noisy Signal')
plt.title('Power Spectral Density (PSD)')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Power/Frequency (dB/Hz)')
plt.grid(True)
plt.legend()
plt.xlim(0, 30)
plt.ylim(1e-5, 1e1)
plt.tight_layout()
plt.show()

```

OUTPUT:



Result:

The solution to the system of equations using the Getting started with files and arrays Aliasing in Real-World Signals was successfully obtained.

Exp. No.: 6	Statistics using Python	Date:
-------------	-------------------------	-------

Statistics of a Noisy Signal

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import skew, kurtosis

# Generate a noisy sine signal
fs = 1000
t = np.linspace(0, 1, fs, endpoint=False)
signal = np.sin(2 * np.pi * 5 * t)
noise = np.random.normal(0, 0.5, size=fs)
noisy_signal = signal + noise

# --- Statistics ---
mean_val = np.mean(noisy_signal)
std_val = np.std(noisy_signal)
rms_val = np.sqrt(np.mean(noisy_signal**2))
skew_val = skew(noisy_signal)
kurt_val = kurtosis(noisy_signal)

# --- Output Statistics ---
print(f'Mean: {mean_val:.4f}')
print(f'Standard Deviation: {std_val:.4f}')
print(f'RMS: {rms_val:.4f}')
print(f'Skewness: {skew_val:.4f}')
print(f'Kurtosis: {kurt_val:.4f}')

# --- Plot Signal & Histogram ---
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(t, noisy_signal)
plt.title("Noisy Signal")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")

plt.subplot(1, 2, 2)
plt.hist(noisy_signal, bins=50, density=True, color='skyblue', edgecolor='black')
plt.title("Signal Amplitude Histogram")
plt.xlabel("Amplitude")
plt.ylabel("Probability Density")

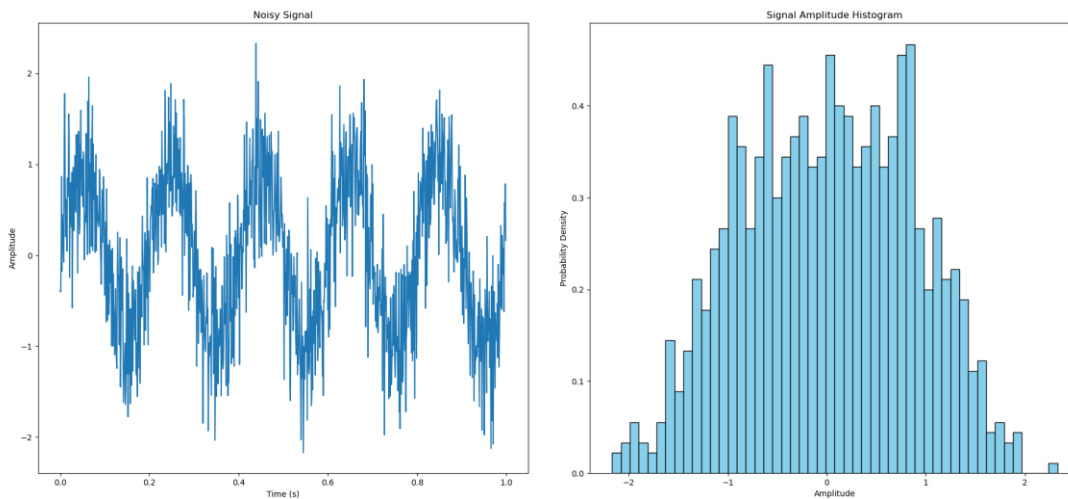
plt.tight_layout()
plt.show()
```


OUTPUT:

```
Mean: 0.0051
Standard Deviation: 0.8595
RMS: 0.8595
Skewness: -0.0583
Kurtosis: -0.7225
```

Interpretation of Output

- Mean ≈ 0 → Balanced around zero
- High std dev → High noise or variability
- RMS → Shows signal energy (good for audio and vibration)
- Skew $\neq 0$ → Asymmetric signal
- Kurtosis > 3 → Heavy tails / outliers



Result:

The solution to the system of equations using the Statistics using Python was successfully obtained

Exp. No.: 07	Linear combination of vectors and Computation of determinant, rank of a matrix	Date:
--------------	--	-------

Aim:

To verify linear combination of vectors using Python.

To compute the determinant and rank of a matrix using Python.

Tools Required:

Python (3.x)

LINEAR COMBINATION OF VECTORS:

Theory:

A vector v is said to be a linear combination of vectors $v_1 + v_2, \dots, v_n$ if

$$v_1 c_1 + v_2 c_2, \dots, v_n c_n$$

Where $c_1, c_2, \dots, c_n$ are scalars (coefficients).

DETERMINANT AND RANK OF A MATRIX

Theory:

- Determinant: A scalar value that can be computed from the elements of a square matrix.
- Rank: The maximum number of linearly independent row or column vectors in the matrix.

Program:

```
import numpy as np

# Define vectors
v1 = np.array([1, 2])
v2 = np.array([3, 4])
c1 = 2
c2 = -1

# Linear combination: v = c1*v1 + c2*v2
v = c1 * v1 + c2 * v2

print("Vector v1:", v1)
print("Vector v2:", v2)
print("Linear combination (2*v1 - 1*v2):", v)

import numpy as np

# Define matrix
A = np.array([[2, 3, 1],
              [4, 7, 7],
              [6, 18, 22]])

# Compute determinant
det = np.linalg.det(A)

# Compute rank
rank = np.linalg.matrix_rank(A)
```

```
print("Matrix A:\n", A)
print("Determinant of A:", round(det, 4))
print("Rank of A:", rank)
```

Sample Input: Linear Combination

$v_1 = [1, 2]$
 $v_2 = [3, 4]$
 $c_1 = 2, c_2 = -1$

Sample Input: Determinant and Rank of A Matrix

Matrix A = $\begin{bmatrix} 2 & 3 & 1 \\ 4 & 7 & 7 \\ 6 & 18 & 22 \end{bmatrix}$

Output:

Linear combination ($2*v_1 - 1*v_2$): $[-1 \ 0]$

Determinant of A: 4.0

Rank of A: 3

Result:

- Verified linear combinations of vectors successfully using Python.
- Computed the determinant and rank of a matrix using NumPy functions.

Exp. No.: 08	Lower - Upper Decomposition of vectors	Date:
--------------	--	-------

Aim:

To write a Python program to solve a system of linear equations using LU Decomposition Method, and to find the solution vector x by decomposing matrix A into a lower triangular matrix L and an upper triangular matrix U .

Tools Required:

Python (3.x)

Theory:

LU Decomposition involves decomposing a matrix A into a product of a Lower Triangular Matrix (L) and an Upper Triangular Matrix (U):

$$A=LU$$

To solve the system:

$$Ax=b$$

We follow:

- Solve $Ly=b \rightarrow$ Forward Substitution
- Solve $Ux=y \rightarrow$ Backward Substitution

Algorithm Steps:

- Accept coefficient matrix A and RHS vector b
- Decompose A into L and U
- Solve $Ly=b$
- Solve $Ux=y$
- Output the solution vector x

Program:

```
import numpy as np
def lu_decomposition(A):
    n = len(A)
    L = np.zeros((n, n))
    U = np.zeros((n, n))
    for i in range(n):
        # Upper Triangular
        for k in range(i, n):
            sum_ = sum(L[i][j] * U[j][k] for j in range(i))
            U[i][k] = A[i][k] - sum_

        # Lower Triangular
        for k in range(i, n):
            if i == k:
                L[i][i] = 1
            else:
                sum_ = sum(L[k][j] * U[j][i] for j in range(i))
```

```

        L[k][i] = (A[k][i] - sum_) / U[i][i]
    return L, U
def forward_substitution(L, b):
    n = len(b)
    y = np.zeros(n)
    for i in range(n):
        y[i] = b[i] - sum(L[i][j] * y[j] for j in range(i))
    return y

def backward_substitution(U, y):
    n = len(y)
    x = np.zeros(n)
    for i in reversed(range(n)):
        x[i] = (y[i] - sum(U[i][j] * x[j] for j in range(i+1, n))) / U[i][i]
    return x

# Main function
A = np.array([[2, 3, 1],
              [4, 7, 7],
              [6, 18, 22]], dtype=float)
b = np.array([1, 2, 3], dtype=float)

L, U = lu_decomposition(A)
y = forward_substitution(L, b)
x = backward_substitution(U, y)
print("Matrix L:\n", L)
print("Matrix U:\n", U)
print("Solution x:\n", x)

```

Output:

Matrix L:
[[1. 0. 0.]
[2. 1. 0.]
[3. 3. 1.]]

Matrix U:
[[2. 3. 1.]
[0. 1. 5.]
[0. 0. 2.]]

Solution x:
[1. 1. 0.]

Result:

The LU Decomposition of the given matrix was successfully computed using Python. The system of linear equations was solved using forward and backward substitution, and the correct solution vector x was obtained.

Exp. No.: 09	Gauss-Seidel Method	Date:
--------------	---------------------	-------

Aim:

To solve a system of linear equations using the Gauss-Seidel Iteration Method

Tools Required:

Python (3.x)

Theory:

The Gauss-Seidel Method is an iterative method used to solve a system of linear equations of the form:

$$Ax=b$$

Where:

- A is a coefficient matrix
- x is the column vector of unknowns
- b is the constant matrix

In Gauss-Seidel, each equation is solved for one variable in terms of others and updated immediately:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right)$$

The process is repeated until the values converge within a desired tolerance.

Convergence Criteria:

The matrix A should preferably be diagonally dominant.

Algorithm Steps:

- Start with an initial guess vector $x^{(0)}$
- For each variable, compute new values using the Gauss-Seidel formula
- Continue iterating until the change in values is less than the specified tolerance
- Display the final values as the solution.

Program:

```
a = [[27, 6, -1], [6, 15, 2], [1, 1, 54]]

# Find diagonal coefficients
diag = np.diag(np.abs(a))

# Find row sum without diagonal
off_diag = np.sum(np.abs(a), axis=1) - diag

if np.all(diag>off_diag):
```

```

print('matrix is diagonally dominant')
else:
print('NOT diagonally dominant')
    x1 = 0
x2 = 0
x3 = 0
epsilon = 0.01
converged = False

x_old = np.array([x1, x2, x3])

print('Iteration results')
print(' k,  x1,  x2,  x3 ')
for k in range(1, 50):
    x1 = (85-36*x2+1*x3)/27
    x2 = (72-6*x1-2*x3)/(15)
    x3 = (110-1*x1-1*x2)/(54)
    x = np.array([x1, x2, x3])
    # check if it is smaller than threshold
    dx = np.sqrt(np.dot(x-x_old, x-x_old))

print("%d, %.4f, %.4f, %.4f"%(k, x1, x2, x3))
    if dx < epsilon:
        converged = True
        print('Converged!')
        break

    # assign the latest x value to the old value
x_old = x

if not converged:
    print('Not converge, increase the # of iterations')

```

Output:

```

matrix is diagonally dominant
Iteration results
k,  x1,  x2,  x3
1, 3.1481, 3.5407, 1.9132
2, -1.5020, 5.1457, 1.9696
3, -3.6398, 5.9933, 1.9935
4, -4.7691, 6.4419, 2.0061
5, -5.3667, 6.6792, 2.0127
6, -5.6829, 6.8048, 2.0163
7, -5.8502, 6.8713, 2.0181
8, -5.9388, 6.9064, 2.0191
9, -5.9856, 6.9250, 2.0196
10, -6.0104, 6.9349, 2.0199
11, -6.0236, 6.9401, 2.0201
12, -6.0305, 6.9429, 2.0201
Converged.

```

Result:

The solution to the system of equations using the Gauss-Seidel method was successfully obtained.

Exp. No.: 10	Solve Systems of Linear Equations in Python	Date:
--------------	---	-------

Aim: To solve a system of linear equations using Python for computing filter coefficients in a finite impulse response (FIR) filter design.

Tools required:

- Python (3.x)

Theory:

In signal processing, FIR filter design often requires solving systems of linear equations to determine filter coefficients that meet specific frequency response requirements. For example, designing a linear-phase filter involves setting up constraints in matrix form $AX=Y$, where A contains sample points of the frequency response, X is the vector of filter coefficients, and Y is the desired output response. Solving such a system provides the optimal filter taps.

Python's **NumPy** library provides efficient methods like `numpy.linalg.solve()` to compute solutions to such equations when the system has a unique solution.

Procedure:

1. Define the coefficient matrix A and the result vector y representing filter constraints.
2. Use `numpy.linalg.solve()` to compute the unknown filter coefficients.
3. Verify the solution by matrix multiplication.

Solve the following equations:

$$\begin{aligned} 4x_1 + 3x_2 - 5x_3 &= 2 \\ -2x_1 - 4x_2 + 5x_3 &= 5 \\ 8x_1 + 8x_2 &= -3 \end{aligned}$$

Program:

```
import numpy as np
```

```
A = np.array([[4, 3, -5],
              [-2, -4, 5],
              [8, 8, 0]])
```

```
y = np.array([2, 5, -3])
```

```
x = np.linalg.solve(A, y)
print(x)
```

```
To verify, 4*x[0] + 3*x[1] - 5*x[2]
```

Output:

```
[ 2.20833333 -2.58333333 -0.18333333]
```

Verification output: 2.0

Result:

The filter coefficients were successfully computed by solving a system of linear equations using Python.

Exp. No.: 11	Eigenvalues and eigenvectors in Python	Date:
--------------	--	-------

Aim:

To compute the eigenvalues and eigenvectors of a matrix using Python and interpret their role in signal transformation and system analysis.

Tools Required:

- Python (3.x)

Theory:

In signal processing, eigenvalues and eigenvectors are crucial in the analysis of linear systems, particularly in **signal decorrelation** and **modal decomposition**. The eigenvectors of a transformation matrix represent the directions along which the signal energy is preserved, while the eigenvalues indicate the scaling (gain or attenuation) along those directions.

- Multiplying an eigenvector v by the matrix A does not change its direction; it changes only its strength by the factor λ .
- The eigenvectors/values contain all of the information in the matrix A (and vice versa).
- Diagonalization by eigendecomposition, $V^{-1}AV = \Lambda$

or, equivalently, $A = V\Lambda V^{-1}$.

Procedure:

1. Define the matrix A for which eigen decomposition is required.
2. Use NumPy's `eig()` function to compute eigenvalues and eigenvectors.
3. Display and interpret the results

Compute the eigenvalues and eigenvectors for matrix $A = \begin{bmatrix} 2 & 2 & 4 \\ 1 & 3 & 5 \\ 2 & 3 & 4 \end{bmatrix}$.

Program:

```
a = np.array([[2, 2, 4],
              [1, 3, 5],
              [2, 3, 4]])
w,v=eig(a)
print('E-value:', w)
print('E-vector', v)
```

Output:

```
E-value: [ 8.80916362  0.92620912 -0.73537273]
E-vector: [[-0.52799324 -0.77557092 -0.36272811]
 [-0.604391   0.62277013 -0.7103262 ]
 [-0.59660259 -0.10318482  0.60321224]]
```

Result:

The eigenvalues and eigenvectors of the given matrix were successfully computed and can be used for signal transformation.

Exp. No.: 12	Generation of basic sequences using Python	Date:
--------------	--	-------

Aim:

To generate and visualize basic discrete-time signal sequences using Python for use in digital signal processing applications.

Tools Required:

- Python (3.x)

Theory:

Basic discrete-time sequences are fundamental in digital signal processing (DSP). They are used for analyzing, modeling, and simulating systems. Some commonly used sequences include:

1. **Unit Impulse ($\delta[n]$)** – A signal that is 1 at $n=0$ and 0 elsewhere.
2. **Unit Step ($u[n]$)** – A signal that is 0 for $n<0$ and 1 for $n\geq 0$.
3. **Ramp Sequence** – A linearly increasing sequence.
4. **Exponential Sequence** – Of the form a^n , used in system stability analysis.
5. **Sinusoidal Sequence** – Used in frequency analysis.

These sequences help in system response analysis, filter design, and are building blocks for more complex signals.

Procedure:

1. Define the discrete-time index range nn .
2. Use NumPy to create the desired sequences.
3. Plot the signals using Matplotlib.
4. Label the plots for visualization.

Program:

```
import numpy as np
import matplotlib.pyplot as plt

n = np.arange(-10, 11, 1)

# Unit Impulse
impulse = np.where(n == 0, 1, 0)

# Unit Step
step = np.where(n >= 0, 1, 0)

# Ramp
ramp = np.where(n >= 0, n, 0)

# Exponential
a = 0.8
expo = a ** n

# Sinusoidal
sin_seq = np.sin(0.2 * np.pi * n)

# Plotting
plt.figure(figsize=(10, 10))

plt.subplot(3, 2, 1)
plt.stem(n, impulse)
plt.title('Unit Impulse  $\delta[n]$ ')
```

```
plt.subplot(3, 2, 2)
plt.stem(n, step)
plt.title('Unit Step  $u[n]$ ')

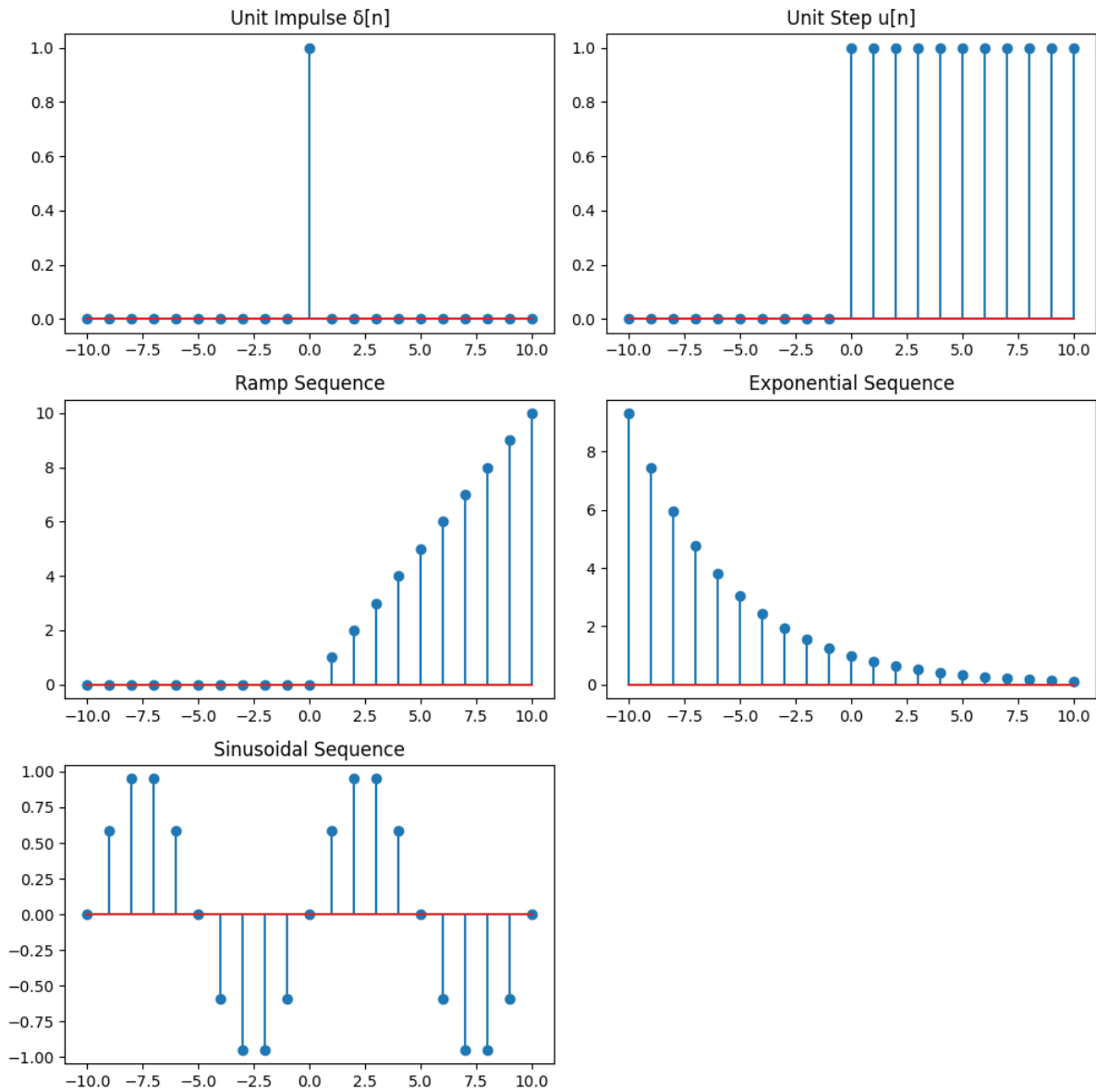
plt.subplot(3, 2, 3)
plt.stem(n, ramp)
plt.title('Ramp Sequence')

plt.subplot(3, 2, 4)
plt.stem(n, expo)
plt.title('Exponential Sequence')

plt.subplot(3, 2, 5)
plt.stem(n, sin_seq)
plt.title('Sinusoidal Sequence')

plt.tight_layout()
plt.show()
```

Output:



Result: Basic discrete-time signal sequences were successfully generated and visualized using Python.

Exp. No.: 13	Spectral analysis of signals	Date:
--------------	------------------------------	-------

Aim: To perform spectral analysis of a noisy musical note using FFT and visualize its frequency components.

Tools required:

Theory:

Spectrum analysis is the process of decomposing a signal into its frequency components and revealing the amplitude of each frequency component present in it. By analyzing the frequency spectrum, it can identify the patterns, anomalies, and other characteristics of the signal present in a signal and their magnitudes. Spectrum analysis can be performed in both the time domain and the frequency domain.

Types of Spectrum Analysis

Broadly Spectrum Analysis can be divided into two parts.

1. *Continuous Spectrum Analysis:* Analyzes continuous-time signals using techniques like Fourier transform.
2. *Discrete Spectrum Analysis:* Analyzes discrete-time signals using techniques like discrete Fourier transform (DFT) and its fast implementation, the fast Fourier transform (FFT).

Principles of Spectrum Analysis

The core principle of spectrum analysis is based on the understanding that any complex, time-varying signal can be represented as a sum of simple sinusoidal waves (sine and cosine functions) of various frequencies, amplitudes, and phases. This representation is made possible through the use of mathematical transformations that convert time-domain signals (signals represented as variations over time) into frequency-domain signals (signals represented as variations over frequency). For example, identifying the dominant frequencies in a piece of music or detecting the frequency of a periodic signal corrupted by noise.

Fast Fourier Transform (FFT)

- The Fourier transform is a mathematical tool used to decompose a signal into its constituent frequencies.
- In Python, the Fourier transform can be computed using libraries like NumPy.
- The resulting spectrum represents the frequency content of the signal.

Procedure:

- 1 Generates musical note signal with added noise, then applies Fast Fourier Transform (FFT) to analyze its frequency spectrum.
- 2 The resulting plot displays the amplitude of frequency components present in the signal.
- 3 Peaks in the plot represent dominant frequencies, indicating the note's fundamental frequency and harmonics.
- 4 The noise introduces additional frequency components, visible as smaller peaks or fluctuations in the plot.

Program:

```
import numpy as np
import matplotlib.pyplot as plt

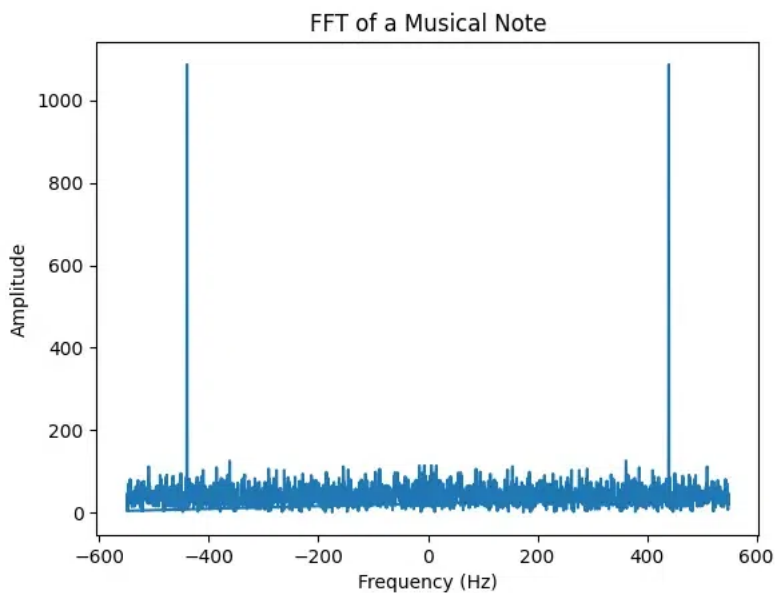
# Generating a sample musical note signal
fs = 1100 # Sampling frequency (Hz)
duration = 2 # seconds
frequency = 440 # A4 note frequency (Hz)
```

```
t = np.linspace(0, duration, int(fs * duration), endpoint=False)
signal = np.sin(2 * np.pi * frequency * t) + np.random.normal(0, 1, len(t)) # Signal with
noise

# Applying FFT
fft_result = np.fft.fft(signal)
freq = np.fft.fftfreq(t.shape[-1], d=1/fs)

# Plotting the spectrum
plt.plot(freq, np.abs(fft_result))
plt.title('FFT of a Musical Note')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Amplitude')
plt.show()
```

Output:



Result:

Spectral analysis using FFT was successfully performed, and the dominant frequency components of the signal were clearly identified.

Exp. No.: 14	Sampling of continuous-time signals	Date:
--------------	-------------------------------------	-------

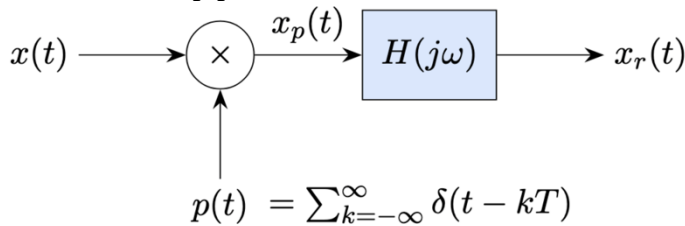
Aim:

To implement the sampling and reconstruction of a continuous-time signal and observe the effects of aliasing with respect to the Nyquist rate.

Tools required:

Theory:

The impulse train sampling and reconstruction of a continuous time signal works. The sampling and reconstruction pipeline is illustrated below.



Here $x(t)$ is the CT input signal to be sampled and reconstructed, and $x_r(t)$ is the reconstruction result. $p(t)$ represents an impulse train and $x_p(t)$ is the samples of $x(t)$. $x_r(t)$ is obtained by low-pass filtering the sampled signal $x_p(t)$. The implementation of these steps, especially sampling a signal with a series of Dirac delta impulses that extend to minus and plus infinities and integrals involving such unlimited signals, is not straightforward and one do not always get the expected results.

The Nyquist rate is the minimum sampling rate required to accurately reconstruct a band-limited signal without any loss of information.

$\omega_s > 2\omega_M$, where ω_M is the bandwidth of the input signal.

Sampling below this rate causes aliasing, where different frequency components overlap, making perfect reconstruction impossible.

Procedure:

1. Define the input continuous-time signal: $x(t) = \cos(\omega_0 * t)$.

2. Choose the Sampling Period (T) and Frequency (ω_s)

Case 1: sample the signal with sampling period $T = 0.1$, with sampling frequency $\omega_s = \frac{2\pi}{0.1} = 20\pi$. Resulting in no aliasing.

Case 2: $\omega_s = \pi$ which corresponds to $T = \frac{2\pi}{\pi} = 2$, results in aliasing.

3. Define a symbolic lambda function $f(n)$ and sample it from $n = -100$ to 100 :

4. Visualize the samples as discrete-time impulses using a stem plot.

5. Use the sinc function for reconstruction:

$$x_r(t) = \sum_{n=-\infty}^{\infty} x_p[n] \operatorname{sinc}\left(\frac{t - nT}{T}\right)$$

6. Plot Reconstructed Signal using plot function

Program:

Case 1: Sampling frequency higher than Nyquist rate.

```
import sympy as sym
t = sym.symbols('t', real=True)
w0 = 2
x = sym.cos(w0*t)
sym.plot(x, (t, 0, 2*sym.pi), ylabel=r'$x(t)$');

# sampling period
T = 0.1
n = sym.symbols('n', integer=True)

# we create a temporary function f, which we will call to get values of x_p
f = sym.Lambda(n, x.subs(t, n*T))
import numpy as np
# here we sample the signal for integer n values from -100 to 100.
# Theoretically, we should do the sampling from minus infinity to plus infinity
# but for practical reasons, we limit this.
ns = np.arange(-100,100)
xp = [f(i) for i in ns]

import matplotlib.pyplot as plt
plt.stem(ns,xp)
plt.xlim([0,50])
plt.xlabel('n')
plt.ylabel(r'$x_p(nT)$');

xr = sum(xp[i]*T*sym.sin((t-ns[i]*T)*sym.pi/T)/(sym.pi*(t-ns[i]*T)) for i in ns)
sym.plot(xr, (t, 0, 2*sym.pi), ylabel=r'$x_r(t)$');
sym.plot(x, (t, 0, 2*sym.pi), ylabel=r'$x(t)$');
```

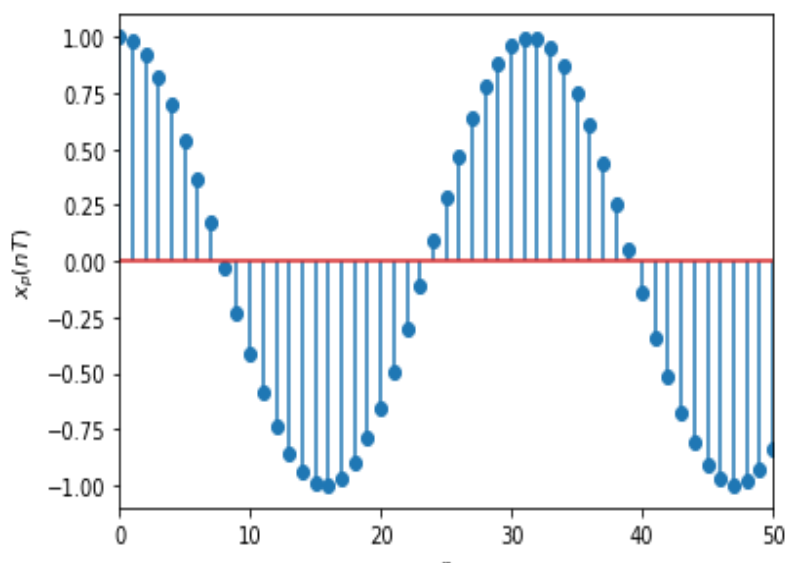
Case 2: Sampling frequency less than Nyquist.

```
# sampling:
T = 2
f = sym.Lambda(n, x.subs(t, n*T))
xp = [f(i) for i in ns]

# reconstruction:
xr = sum(xp[i]*T*sym.sin((t-ns[i]*T)*sym.pi/T)/(sym.pi*(t-ns[i]*T)) for i in ns)
sym.plot(xr, (t, 0, 2*sym.pi), ylabel=r'$x_r(t)$');
sym.plot(x, (t, 0, 2*sym.pi), ylabel=r'$x(t)$');
```

Output: (left)

Sampled Signal:



	Original signal	Reconstructed Signal
Case 1		
Case 2		

Result:

We have successfully implemented the sampling and reconstruction of a continuous-time cosine signal. The results confirm that proper sampling allows perfect reconstruction, while undersampling leads to aliasing.